Лабораторная работа №26

Чат объявления: отображение и отправка сообщений

О чем лабораторная?

В этой работе добавим чат, связанный с конкретным объявлением. Пользователь сможет открыть экран чата, увидеть сообщения и отправить новое сообщение автору объявления.

В API сообщения хранятся не в Android-приложении, а на сервере. Поэтому после открытия экрана приложение загружает историю через API.

Главное

- история сообщений загружается через `GET /items/{item_id}/messages`
- новое сообщение отправляется через `POST /items/{item_id}/messages`
- сообщение отправляется только авторизованным пользователем
- сервер ограничивает длину сообщения 500 символами
- после отправки список сообщений нужно обновить

Часть 1. Изучите API сообщений

Откройте документацию и найдите операции:

- `GET /items/{item_id}/messages`
- `POST /items/{item_id}/messages`

Для загрузки истории передаются:

- `item_id` - идентификатор объявления
- `page` - номер страницы
- `page_size` - количество сообщений на странице

Пример ответа:

```
{
  "items": [
    {
      "id": 1,
      "item_id": 10,
      "user": {
        "id": "UUID",
        "username": "student",
        "about_me": null
      },
      "text": "Здравствуйте! Объявление еще актуально?",
      "created_at": "2026-11-02T12:30:00Z"
    }
  ],
```

```
  "meta": {
    "total": 1,
    "page": 1,
    "page_size": 20,
    "total_pages": 1
  }
}
```

Тело запроса для отправки сообщения:

```
{
  "text": "Да, объявление актуально"
}
```

Текст должен содержать от 1 до 500 символов.

Часть 2. Создайте модели сообщений

Создайте `MessageCreate`:

```
public class MessageCreate {
    private final String text;

    public MessageCreate(String text) {
        this.text = text;
    }
}
```

Создайте `MessageResponse`:

```
public class MessageResponse {
    private int id;
    private int item_id;
    private UserResponse user;
    private String text;
    private String created_at;

    public int getId() {
        return id;
    }

    public UserResponse getUser() {
        return user;
    }

    public String getText() {
        return text;
    }
```

```
    public String getCreatedAt() {
        return created_at;
    }
}
```

Для списка сообщений используйте уже знакомую модель:

```
PaginatedResponse<MessageResponse>
```

Она содержит список `items` и данные о страницах в поле `meta`.

Часть 3. Добавьте методы в ApiService

```
@GET("items/{item_id}/messages")
Call<PaginatedResponse<MessageResponse>> getMessages(
        @Path("item_id") int itemId,
        @Query("page") int page,
        @Query("page_size") int pageSize
);

@POST("items/{item_id}/messages")
Call<MessageResponse> createMessage(
        @Path("item_id") int itemId,
        @Body MessageCreate request
);
```

Часть 4. Создайте экран чата

Добавьте Activity `ChatActivity`.

На экране разместите:

- кнопку "Назад"
- название объявления или его идентификатор
- `RecyclerView` для списка сообщений
- `EditText` для ввода текста
- кнопку "Отправить"
- `ProgressBar` на время загрузки

Создайте разметку одного сообщения `row_message.xml`. В ней должны быть:

- имя пользователя
- текст сообщения
- дата создания

Создайте `MessageAdapter`, который получает `List<MessageResponse>` и заполняет эту разметку.

Часть 5. Откройте чат из деталей объявления

В `DetailsActivity` уже есть идентификатор загруженного объявления. Передайте его в `ChatActivity`:

```
Intent intent = new Intent(this, ChatActivity.class);
intent.putExtra("item_id", loadedItem.getId());
startActivity(intent);
```

В `ChatActivity` получите идентификатор:

```
int itemId = getIntent().getIntExtra("item_id", -1);

if (itemId == -1) {
    finish();
    return;
}
```

Без `item_id` приложение не сможет понять, к какому объявлению относится чат.

Часть 6. Загрузите историю сообщений

После открытия `ChatActivity` выполните запрос:

```
api.getMessages(itemId, 1, 20).enqueue(
        new Callback<PaginatedResponse<MessageResponse>>() {
            @Override
            public void onResponse(
                    Call<PaginatedResponse<MessageResponse>> call,
                    Response<PaginatedResponse<MessageResponse>> response
            ) {
                if (response.isSuccessful() && response.body() != null) {
                    List<MessageResponse> messages =
                            response.body().getItems();
                    recyclerView.setAdapter(new MessageAdapter(messages));
                } else {
                    showMessage("Не удалось загрузить сообщения");
                }
            }

            @Override
            public void onFailure(
                    Call<PaginatedResponse<MessageResponse>> call,
                    Throwable t
            ) {
                showMessage("Ошибка сети");
            }
```

```
        }
);
```

Часть 7. Отправьте сообщение

Перед отправкой проверьте текст:

```
String text = messageEdit.getText().toString().trim();

if (text.isEmpty()) {
    messageEdit.setError("Введите сообщение");
    return;
}

if (text.length() > 500) {
    messageEdit.setError("Не более 500 символов");
    return;
}
```

Отправьте сообщение:

```
api.createMessage(itemId, new MessageCreate(text)).enqueue(
        new Callback<MessageResponse>() {
            @Override
            public void onResponse(
                    Call<MessageResponse> call,
                    Response<MessageResponse> response
            ) {
                if (response.isSuccessful() && response.body() != null) {
                    messageEdit.setText("");
                    loadMessages();
                } else if (response.code() == 401) {
                    showMessage("Сначала войдите в аккаунт");
                } else {
                    showMessage("Не удалось отправить сообщение");
                }
            }

            @Override
            public void onFailure(Call<MessageResponse> call, Throwable t) {
                showMessage("Ошибка сети");
            }
        }
);
```

После успешной отправки очистите поле и снова загрузите историю. Так в списке появится сообщение с данными, которые вернул сервер.

Часть 8. Самостоятельная работа

Доработайте чат в своем проекте:

1. откройте чат с экрана деталей объявления
2. загрузите историю сообщений через API
3. покажите автора, текст и время сообщения
4. добавьте отправку нового сообщения
5. обработайте пустой список, ошибку сети и ответ `401`
6. после отправки обновите список сообщений

Требования

- экран чата открывается для конкретного объявления
- история сообщений загружается через `GET /items/{item_id}/messages`
- сообщения отображаются в `RecyclerView`
- новое сообщение отправляется через `POST /items/{item_id}/messages`
- длина сообщения проверяется до отправки
- после отправки список обновляется

Дополнительно (по желанию)

1. Показывайте свои сообщения и сообщения других пользователей разными цветами
2. Прокручивайте список к последнему сообщению после отправки